Menu Driven Doubly Linked List Program (Deletion)

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};

/* Create Doubly Linked List */
struct Node* create(struct Node *head)
{
    int n, i, value;
    struct Node *newnode, *temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        newnode = (struct Node*)malloc(sizeof(struct Node));
        printf("Enter data: ");
        scanf("%d", &value);

        newnode->data = value;
        newnode->prev = NULL;
        newnode->next = NULL;

        if(head == NULL)
        {
            head = newnode;
            temp = newnode;
        }
        else
        {
            temp->next = newnode;
            newnode->prev = temp;
            temp = newnode;
        }
    }
    return head;
}
```

```
/* Delete at Beginning */
struct Node* deleteBegin(struct Node *head)
{
    struct Node *temp;

    if(head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    temp = head;
    head = head->next;

    if(head != NULL)
    {
        head->prev = NULL;
    }

    free(temp);
    return head;
}

/* Delete at End */
struct Node* deleteEnd(struct Node *head)
{
    struct Node *temp;

    if(head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    temp = head;

    // Only one node
    if(temp->next == NULL)
    {
        free(temp);
        return NULL;
    }

    // Traverse to last node
    while(temp->next != NULL)
    {
        temp = temp->next;
    }
```

```
    temp->prev->next = NULL;
    free(temp);

    return head;
}

/* Delete at Specific Position */
struct Node* deletePosition(struct Node *head)
{
    struct Node *temp;
    int pos, i;

    if(head == NULL)
    {
        printf("List is empty\n");
        return head;
    }

    printf("Enter position: ");
    scanf("%d", &pos);

    temp = head;

    // Delete first node
    if(pos == 1)
    {
        head = deleteBegin(head);
        return head;
    }

    // Traverse to position
    for(i = 1; i < pos && temp != NULL; i++)
    {
        temp = temp->next;
    }

    if(temp == NULL)
    {
        printf("Invalid position\n");
        return head;
    }

    // If last node
    if(temp->next == NULL)
    {
        temp->prev->next = NULL;
    }
    else
    {
```

```
        temp->prev->next = temp->next;
        temp->next->prev = temp->prev;
    }

    free(temp);
    return head;
}

/* Display Doubly Linked List */
void display(struct Node *head)
{
    struct Node *temp;

    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;

    printf("Linked List: ");
    while(temp != NULL)
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

/* Main Function */
int main()
{
    struct Node *head = NULL;
    int choice;

    while(1)
    {
        printf("\n----- MENU -----\n");
        printf("1. Create\n");
        printf("2. Delete at Beginning\n");
        printf("3. Delete at End\n");
        printf("4. Delete at Position\n");
        printf("5. Display\n");
        printf("6. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);
```

```
        switch(choice)
        {
            case 1:
                head = create(head);
                break;

            case 2:
                head = deleteBegin(head);
                break;

            case 3:
                head = deleteEnd(head);
                break;

            case 4:
                head = deletePosition(head);
                break;

            case 5:
                display(head);
                break;

            case 6:
                exit(0);

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

Example Run

```
----- MENU -----
1. Create
2. Delete at Beginning
3. Delete at End
4. Delete at Position
5. Display
6. Exit

Enter choice: 1
Enter number of elements: 3
Enter data: 10
Enter data: 20
Enter data: 30
```

```
Enter choice: 2
Enter choice: 5
Linked List: 20 <-> 30 <-> NULL
```